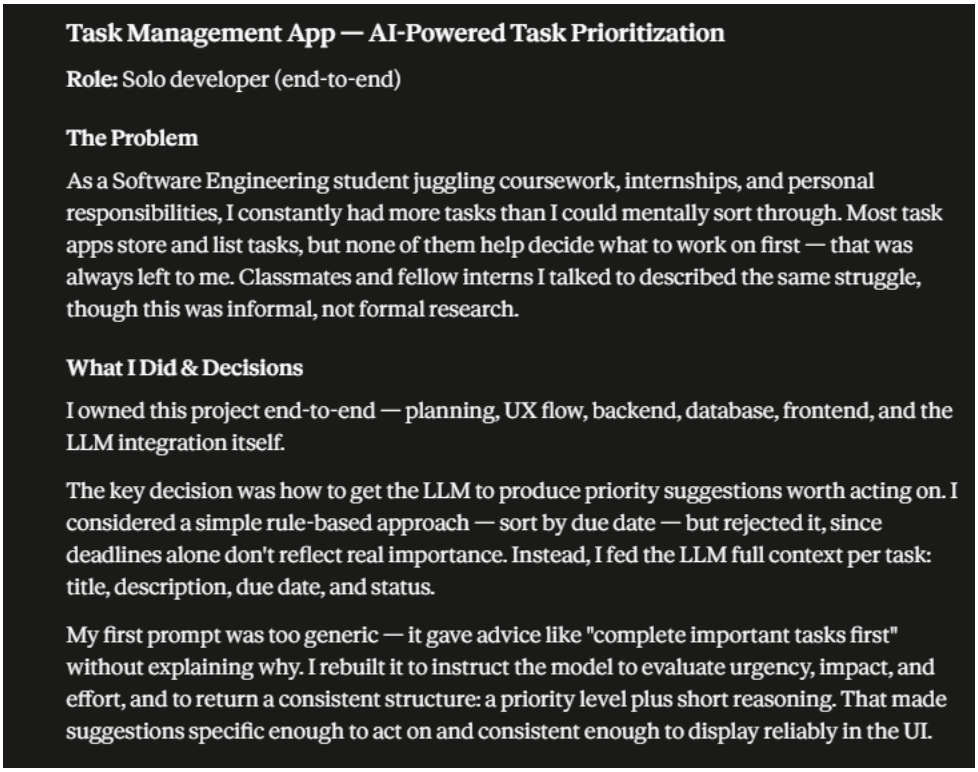
WEEK 2 ASSIGNMENT AI FLUENCY

Voice Card:

Direct, clear, practical, professional, honest, no buzzwords.



Task Management App — AI-Powered Task Prioritization

Role: Solo developer (end-to-end)

The Problem

As a Software Engineering student juggling coursework, internships, and personal responsibilities, I constantly had more tasks than I could mentally sort through. Most task apps store and list tasks, but none of them help decide what to work on first — that was always left to me. Classmates and fellow interns I talked to described the same struggle, though this was informal, not formal research.

What I Did & Decisions

I owned this project end-to-end — planning, UX flow, backend, database, frontend, and the LLM integration itself.

The key decision was how to get the LLM to produce priority suggestions worth acting on. I considered a simple rule-based approach — sort by due date — but rejected it, since deadlines alone don't reflect real importance. Instead, I fed the LLM full context per task: title, description, due date, and status.

My first prompt was too generic — it gave advice like "complete important tasks first" without explaining why. I rebuilt it to instruct the model to evaluate urgency, impact, and effort, and to return a consistent structure: a priority level plus short reasoning. That made suggestions specific enough to act on and consistent enough to display reliably in the UI.

Challenges

AI output reliability. Early responses were inconsistent and hard to render predictably. Fixed by defining a strict output structure — priority level plus reasoning — instead of letting the model respond freely.

External AI dependency. The LLM is an external service, so a failed or slow request couldn't be allowed to break the app. I designed the flow so the task list still works normally without AI suggestions if the request fails — the app degrades gracefully instead of depending on the AI call succeeding.

Outcome

This was a prototype, built solo without formal user testing — the outcome below reflects that scope.

I built a working end-to-end app: task management with real LLM-based prioritization, backend to frontend. I used it myself to plan my own work and see how AI suggestions actually held up in daily use. What I can stand behind is a functional AI-integrated application and real hands-on experience with prompt design, response handling, and full-stack integration.

What I Learned

I started by focusing on getting the LLM to generate suggestions. What actually mattered was whether those suggestions helped someone decide quickly — the AI output only works if it fits how the user actually thinks, not just because it exists. Next time, I'd define what a useful AI output looks like from the user's side first, then build the prompt and interface around that, and test AI responses earlier instead of waiting until integration.

About me:

I am a Software Engineering student focused on building AI-powered applications and backend systems. I enjoy combining engineering skills with user-focused thinking to create practical solutions that solve real problems.

CTA

Interested in discussing AI engineering or software development opportunities?

Use my contact form to get in touch.

Before (generic AI line)

I am a passionate software engineer who builds innovative AI solutions to transform user experiences and improve productivity.

After(my edited version)

I built a task management app with an LLM-based prioritization feature that helps users decide what to work on first by analyzing task context instead of relying only on deadlines.